Smart Display · Group 5א · IoT project 236333, Technion

Feasibility Tests: Communication, Database, UI

Meetin #4 submission: the options we checked before building, what we tried, and what we chose

Shada Nujeidat, Yuval Carmelli, Liat Rokach · 11 October 2026

Three parts of the project carried the most risk: getting a spoken command from a phone to the display (**communication**), deciding where the data lives so the display keeps working without internet (**database**), and building a good-looking touch interface on a small ESP32 board (**UI**). For each one we list the options, what we tried, the result, and the decision. Numbers marked "build" come from compiling the firmware; timings on the board are measured with the HW unit tests (see HW Unit Tests Report).

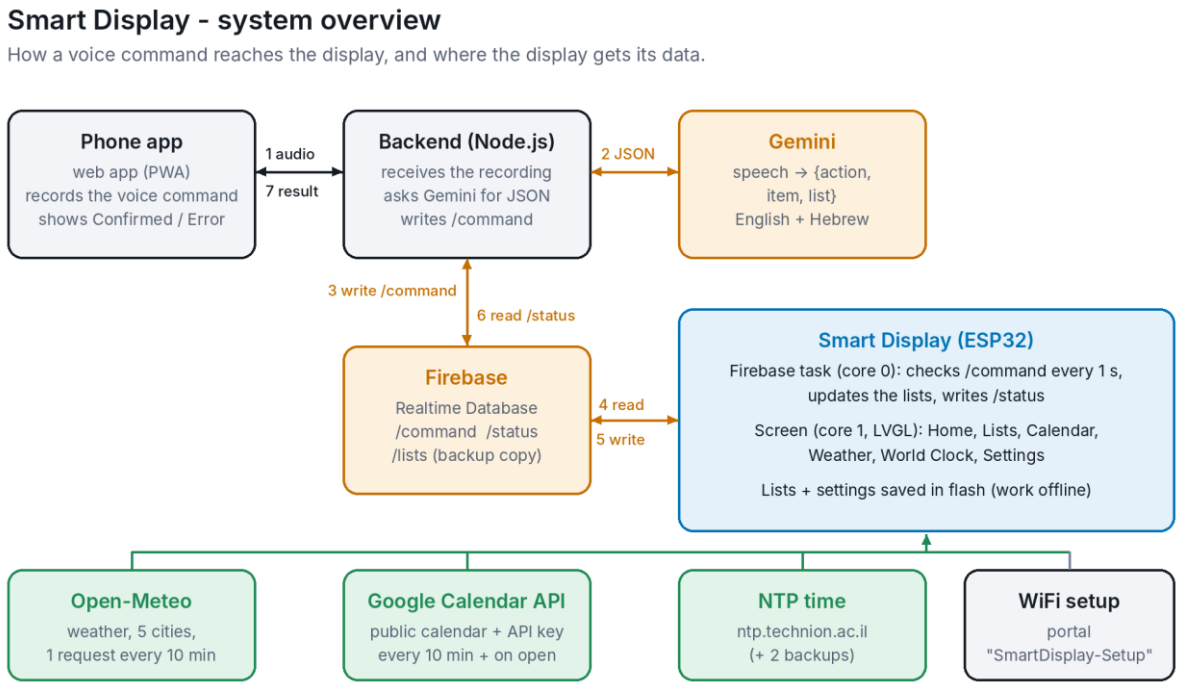


Figure 1. System overview: how a voice command reaches the display, and where the display gets its data

1. Communication

1.1 Voice input: how a spoken command becomes a list change

Option	What we found	Feasible?	Decision
Google Assistant custom commands	Conversational Actions, the only way to give Assistant free-form custom commands ("add milk to my list"), were shut down by Google in June 2023. Smart Home actions only support fixed device traits (on/off, brightness)	No	Dropped
Phone app + Gemini	A web app (PWA) records the command; a small Node.js backend sends the audio to Gemini, which returns JSON `{action, item, list}` in English or Hebrew. Tried end to end: add / remove on both lists works	Yes	**Chosen**
Speech recognition on the ESP32	No microphone on the CYD; on-device recognition of free text is beyond the ESP32	No	Dropped

Why a backend between the app and Gemini: the Gemini API key and the Firebase admin key stay on the server and never reach the phone or the ESP32, and the server checks Gemini's JSON (valid action, known list, non-empty item) before anything is written. Found in testing: Gemini sometimes put to-do items ("walking") on the shopping list, so we added explicit rules for choosing the list to the prompt.

1.2 Transport: phone → display

The phone and the display are often on different networks (phone on mobile data, display on home WiFi), so the message has to go through the internet. The instructor asked us to evaluate MQTT as a lighter alternative to Firebase, since we use only a small part of Firebase.

Criterion	Firebase Realtime DB (REST)	MQTT broker	Web server on the ESP32	Bluetooth (BLE)
Works across networks	Yes	Yes	No: same WiFi only	No: ~10 m range
Delay	Up to 1 s (display checks every second)	Instant (push)	Instant	Instant
Reply to the app	/status path	Needs a second topic	HTTP reply	Notify characteristic
ESP32 code	HTTPClient + ArduinoJson, already used for weather	Extra MQTT library, TLS on port 8883	WebServer library	BLE stack, large
Phone side (web app)	Firebase SDK in the backend	MQTT over WebSockets	Needs the display's IP	Web Bluetooth not on iPhone
Login / security	Email login + database rules	Broker user + password	Open on the LAN	Pairing
Keeps a copy of the lists	/lists	Retained messages only	No	No
Cost on the ESP32	One small request per second; one TLS connection (~40 KB RAM) kept open	One idle connection	Low	BLE + WiFi together use a lot of RAM

Decision: Firebase Realtime Database over plain HTTPS (REST), polled once a second. It works across networks, gives the app a reply and keeps a backup of the lists, with no extra library on the ESP32. A one-second delay is not noticeable for a list command (the app takes longer to record and run Gemini). MQTT would remove the polling and is the documented alternative: all Firebase code is in one file (`firebase_sync.h`) running in its own task, so it can be replaced without touching the screens.

Result: voice commands work end to end: the display applies the command within the one-second check and the app shows the reply ("Added milk to your shopping list"). Read time on the board: HW unit test 8 (`test_firebase`).

1.3 Weather, calendar and time

Data	Options compared	Decision and why
Weather	OpenWeatherMap (API key, one city per request) vs Open-Meteo (no key, several locations per request)	**Open-Meteo**: one HTTPS request returns current weather and a 4-day forecast for all 5 cities, so switching city needs no network and only one secure connection is opened every 10 min. Tested: HW unit test 7

Data	Options compared	Decision and why
Calendar	Google Calendar API with OAuth (user login, tokens on the ESP32) vs a **public** calendar read with an API key vs the private iCal address (download and parse an .ics file)	**Public calendar + API key**: a single GET with `fields=` returns a small JSON with only title, start and end. Trade-off: the calendar is public, so the display uses its own calendar, not a personal one. The private iCal address is the option if this matters. Tested: HW unit test 9
Time	pool.ntp.org only vs the Technion NTP server first	**ntp.technion.ac.il first**, pool.ntp.org and time.google.com as backups: outside time servers are blocked on the Technion network. Israel's summer/winter time rule built in. Tested: HW unit test 6

2. Database and storage

2.1 Where each piece of data lives

Data	Stored in	Why
Shopping and to-do lists	Display flash (NVS), one 2 KB block; copy in Firebase `/lists`	The display is the source of truth, so the lists work without internet. Every change raises a version number; the Firebase copy is updated whenever it differs, so a change made offline is sent once WiFi is back (instructor feedback)
Latest voice command	Firebase `/command`	`{type, item, list, runningCount}`. The number goes up by one per command, so the display can tell a new command from one it already handled
Reply to the app	Firebase `/status`	`{runningCount, success, text}`; the app waits up to 60 s for the reply with its number
Settings (brightness, 24-hour time, city, dark mode, power saving)	Display flash (NVS), namespace "display"	Saved only when a value changes (not while a slider moves) to avoid wearing out the flash; each value is limited to its valid range when loaded
Weather, calendar events	Display RAM	Downloaded again after a restart; shown with "Updated HH:MM" so old data is recognisable
WiFi network	ESP32 WiFi settings (written by the setup portal)	Set from a phone through `SmartDisplay-Setup`, no re-flashing
Secrets (Firebase login, calendar key)	`main/secrets.h`, not in Git	Template `secrets.example.h` in Git; `.gitignore` keeps the real file out

2.2 Options considered

- **Firebase Realtime Database vs Cloud Firestore:** the Realtime Database is a plain JSON tree read with a simple REST URL (`/command.json`), which suits a small ESP32 client. Firestore's REST replies wrap every field in a type object and need more parsing.
- **Firebase library vs plain REST:** the Firebase Arduino libraries are large and add their own task and buffers. Plain HTTPS with the existing HTTPClient and ArduinoJson keeps the program smaller and under our control (timeouts, retries).
- **Security:** database rules allow only signed-in users; the display signs in as its own user; the backend uses the Admin SDK with a service-account key that never leaves the server.
- **Result:** lists survive restarts and power cuts, and offline changes reach Firebase after reconnecting. Tested: HW unit test 4 (`test_storage`, including the 2 KB block and a reset) and test 8 (`test_firebase`).

3. User interface

3.1 Graphics library

Option	What we found	Decision
TFT_eSPI only	Small and fast, ideal to bring up the board. Every button, scroll and text layout has to be drawn by hand; no anti-aliased fonts or right-to-left text	Used for the first version to get all features working
LVGL 9 (on top of TFT_eSPI)	Ready widgets (buttons, sliders, switches, scrolling lists, dropdowns, round scales for the world clock), shared styles for a dark and a light theme, anti-aliased fonts and right-to-left (BiDi) text for Hebrew	**Chosen** for the final UI. The data modules (weather, calendar, lists, Firebase) contain no drawing code, so they were reused unchanged

3.2 Will it fit? Measurements

Measurement	Value	Conclusion
Program size (build)	1,606,735 bytes	51 % of the 3 MB "Huge APP" partition. Does not fit the default 1.3 MB partition, so the README and `sketch.yaml` select Huge APP
RAM for global variables (build)	72,772 bytes (22 %)	Leaves room for the secure connections created at run time (~40 KB each)
LVGL draw buffer	1/10 of the screen, 15 KB	Larger buffers save only a few ms per full redraw and take RAM that TLS needs
Full-screen redraw	≈ 22 ms at 55 MHz SPI (320 × 240 × 16 bit)	Fast enough for smooth scrolling. Checked on the board with HW unit test 1
Clock digits font	Ubuntu Medium 38 px, digits only, ~4 KB	Large clean clock without the cost of a full large font
Hebrew fonts	14 px and 20 px, Hebrew letters only, falling back to Montserrat for Latin	Hebrew list items and events display correctly instead of "???"

3.3 Design decisions

- **Touch targets at least 44 px**, since a resistive screen is less precise than a phone; checked with HW unit test 2.
- **One design system** (`theme.h`): every color and text style is shared, so switching between dark and light mode restyles all screens without rebuilding them.
- **Status on every screen:** a header with back button, title, clock and WiFi/Offline indicator.
- Each screen was rendered on a PC from the same LVGL code and reviewed (layout, alignment, Hebrew, both themes) before flashing. Screens below.

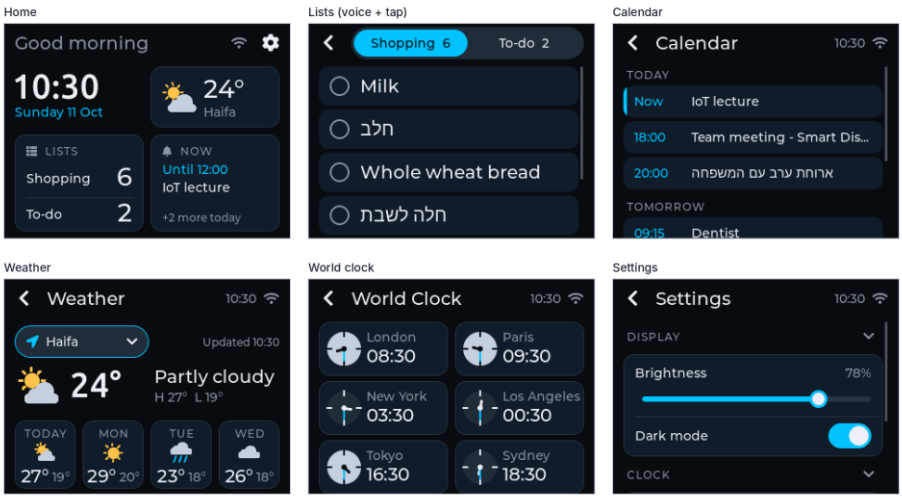


Figure 2. Final screens, dark mode

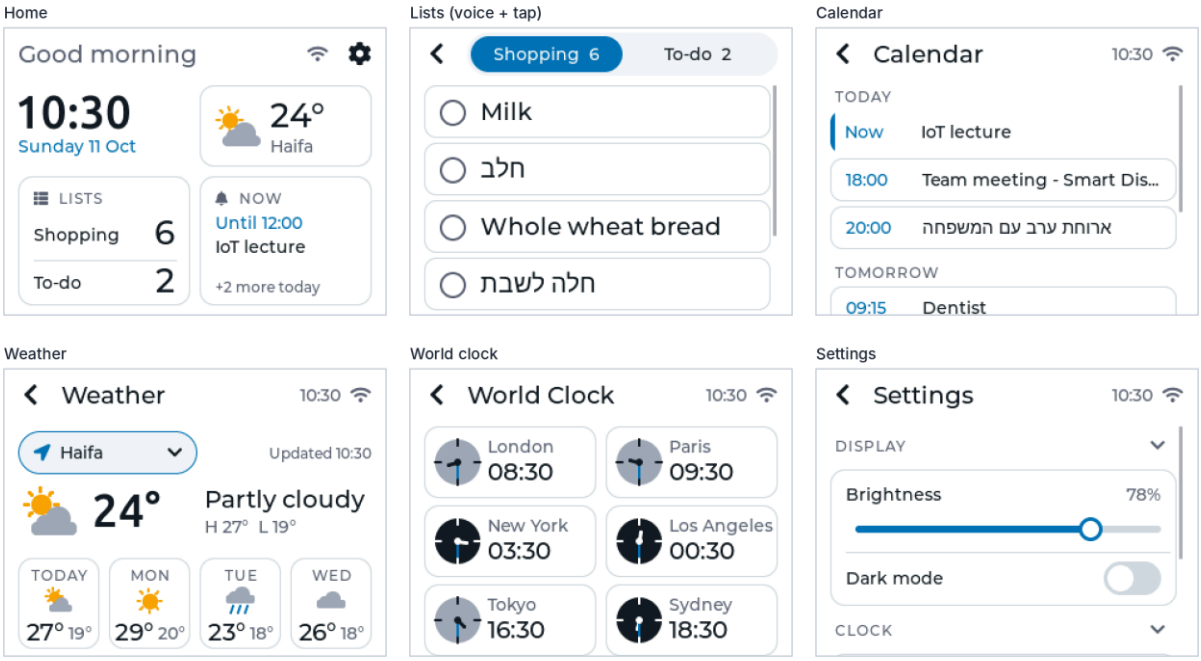


Figure 3. Final screens, light mode

3.4 User testing

We tested the interface with family and friends who were not part of the project: they used the display and the voice app on their own while we watched where they hesitated. Together with our own design reviews, this led to these changes:

- **Weather city:** the city choice was not noticed as something you can tap, so it became a clear button with a location icon and an arrow.
- **Settings:** the screen was long to scroll, so it was split into sections (Display, Clock, Power saving) that open and close.
- **Calendar:** event text was made smaller and the day headings clearer, so more events fit and the screen is easier to scan.
- **Home clock:** digits re-aligned with the rest of the screen and given a clearer font.

- **Light mode:** looked grey on the panel, so it became pure white with outlined rows.

4. Hardware choice

ESP32-2432S028R ("Cheap Yellow Display") instead of an ESP32 board with a separate screen: screen, touch controller, USB and power are on one board, so there is no wiring to come loose and the enclosure is a simple box. The ESP32 has two cores, which lets the Firebase task run on one core while the screen runs on the other. Limits we accepted: 2.4 GHz WiFi only, resistive touch, and the screen and touch on two separate SPI buses (which also means they never block each other).

5. Summary

Area	Decision	Status
Voice input	Phone web app → Node.js backend → Gemini (JSON)	Working end to end
Phone → display	Firebase Realtime Database, REST, checked every 1 s	Working; MQTT documented as the alternative
Weather / calendar / time	Open-Meteo (1 request, 5 cities) / public Google Calendar + API key / Technion NTP	Working
Storage	Lists and settings in flash, lists copied to Firebase, offline changes synced later	Working
UI	LVGL 9 on TFT_eSPI, dark + light theme, Hebrew	Working
Performance	Weather and calendar downloads still run on the screen's core (short pause every 10 min)	Planned: move to the background task